

hassan-sessions.md

Hassan Alloush — Training Sessions

Session 1 — Why an API Layer? `apiRequest<T>`

Learning Objectives

By the end of this session, the team should understand:

- Why we do not call `fetch` directly inside components.
- What `apiRequest<T>` does and why it is generic.
- How the project centralizes request logic.

1. The Problem

Every page in Mystery Room needs data from the server. If each page called `fetch` directly, the base URL, error handling, and response parsing would be repeated in six different files. Changing the API base URL would mean editing six places.

2. Familiar Idea

In earlier sessions, we learned that a function can group repeated code. `updateStats()` grouped counting. `renderTasks()` grouped rendering.

The same idea applies to API calls. We can group the repeated parts into one function.

3. The New Concept

An **API layer** is a small module that owns all communication with the server. Components call named functions; they do not know the base URL, the request method, or how errors are handled.

`apiRequest<T>` is a **generic** function. `T` is a placeholder for "the type of data this request returns." When you call `apiRequest<Mystery[]>("/mysteries")`, TypeScript knows the result is an array of `Mystery`.

4. Tiny Example

```
async function getText(url: string): Promise<string> {  
  
    const response =  
    await fetch(url);  
  
    return  
    response.text();  
  
}  
  
const data = await getText("/hello"); // data:  
string
```

The same shape, but with a type parameter, gives us typed data.

5. Project Application

File: `client/src/api.ts`

```
import type { AnswerResponse, Clue, HintResponse, Mystery }
from "./types";

async function apiRequest<T>(path: string, options?:
RequestInit): Promise<T> {

    const response =
await fetch(`/api/mysteries${path}`, options);

    if (!response.ok) {

        const data = await
response.json().catch(() => null);

        const message =

            data &&
typeof data.error === "string"

                ? data.error

                : "The
request failed. Please try again.";

        throw new
Error(message);

    }

    return (await
response.json()) as T;

}
```

6. Code Walkthrough

The signature

```
async function apiRequest<T>(path: string, options?:
RequestInit): Promise<T>
```

What it does: Declares a generic async function.

Why it exists: `T` lets callers say what type they expect back.

What you see: When called as `apiRequest<Mystery[]>`, `T` becomes `Mystery[]`.

In Plain English: "A request function whose return type is chosen by the caller."

Read it as: "Take a path, return a Promise of T."

The fetch call

```
const response = await fetch(`/api/mysteries${path}`,  
options);
```

What it does: Sends the HTTP request to the server.

Why it exists: The base path `/api/mysteries` is in one place.

What you see: The Network tab shows the request.

In Plain English: "Ask the server for this path."

Read it as: "Template the base URL and append the path."

The `response.ok` check

```
if (!response.ok) { ... }
```

What it does: Detects HTTP error statuses (400, 404, 409, 413, etc.).

Why it exists: `fetch` does not throw on 400 or 404. It only throws on network failure.

What you see: Non-ok responses become `Error` objects.

In Plain English: "If the server did not say OK, throw."

Read it as: "Check the status code."

The error message fallback

```
const data = await response.json().catch(() => null);

const message =

  data &&
  typeof data.error === "string"

    ? data.error

    : "The
request failed. Please try again.";

throw new Error(message);
```

What it does: Tries to read the server's error message. Falls back to a generic one.

Why it exists: The server sends useful errors like `"No mystery found with id 999."`.

What you see: The page displays the server's message when available.

The success return

```
return (await response.json()) as T;
```

What it does: Parses the JSON and tells TypeScript to trust the type.

Why it exists: TypeScript cannot verify the shape of network data. `as T` is a promise, not a check.

What you see: The caller receives typed data.

In Plain English: "Parse the JSON and pretend it is T."

7. Common Mistakes

WRONG: Fetching directly in a component.

```
useEffect(() => {  
  
  fetch("/api/mysteries").then(r =>  
    r.json()).then(setMysteries);  
  
}, []);
```

This skips error handling and does not check `response.ok`.

CORRECT:

```
useEffect(() => {  
  
  getMysteries().then(setMysteries).catch(setError);  
  
}, []);
```

WRONG: Assuming `fetch` throws on 404.

`fetch` throws only on network failure. For 404, you must check `response.ok`.

WRONG: Trusting `as T` at runtime.

`as T` is a promise to TypeScript. It does not validate the data. If the server sends the wrong shape, the app will misbehave.

8. Important Decisions

- **Why a single `apiRequest`?** One place for base URL, error handling, JSON parsing.
- **Why generic `<T>`?** Each caller knows what shape it expects.
- **Why throw on non-ok?** So pages can catch and show errors uniformly.
- **Why `encodeURIComponent` in the helpers?** To safely place IDs in URL paths.

9. Project Flow

Page calls `getMysteryById("1")`

|

▼

`apiRequest<Mystery>("/1")`

|

▼

`fetch("/api/mysteries/1")`

|

▼

Vite proxy forwards to `localhost:3001`

|

▼

Express returns public JSON

|

▼

`apiRequest` checks `response.ok`, parses JSON

|

▼

Page receives `Mystery` object

10. Review Questions

1. Why not call `fetch` inside each component?

2. What does `<T>` mean in `apiRequest<T>` ?
3. What does `response.ok` check?
4. What does `as T` actually do?
5. Why does the project use `encodeURIComponent` ?

11. Glossary

| Term | Meaning |

|---|---|

| API layer | A module that centralizes communication with the server. |

| Generic | A type parameter such as `T`, supplied by the caller. |

| `response.ok` | `true` for status 200–299, `false` otherwise. |

| Type assertion | Telling TypeScript to treat a value as a specific type. |

12. What You Should Be Able to Explain Now

- Why an API layer exists.
- What `apiRequest<T>` does.
- Why `response.ok` must be checked.
- Why `as T` is not validation.

13. Not Yet

- Types for API responses — Session 2.
- Loading, error, and empty states — Session 3.

Session 2 — TypeScript Types for API Responses

Learning Objectives

By the end of this session, the team should understand:

- Why we describe API data with interfaces.
- What shape each type has.
- Why the public `Mystery` type excludes answers.

1. The Problem

When the server sends JSON, TypeScript sees `any`. It cannot help us if we typo a property name, use the wrong type, or expect a field that does not exist.

We need to describe the shape of each response so TypeScript can check our code.

2. Familiar Idea

In earlier sessions, we used TypeScript interfaces to describe the shape of objects: `Task`, `User`, `Opportunity`.

The same idea applies to API responses.

3. The New Concept

An **interface** is a named description of an object's shape — which properties it has, and what type each one is.

When we say `apiRequest<Mystery[]>`, we tell TypeScript that the result is an array of objects matching the `Mystery` interface.

4. Tiny Example

```
interface Point {  
  x: number;  
  y: number;  
}  
  
const p: Point = { x: 1, y: 2 };  
  
p.x = "hello"; // TypeScript error: string not  
assignable to number
```

5. Project Application

File: `client/src/types.ts`

```
export interface Mystery {

    id: number;

    slug: string;

    title: string;

    intro: string;

    totalStages: number;

    currentStage:
number;

    solved: boolean;

    hintsUsed: number;

    hintsRemaining:
number;

    reveal: string |
null;

    currentQuestion:
string;

    currentOptions:
string[];

}

export interface Clue {

    stage: number;

    text: string;

}

export interface AnswerResponse {

    correct: boolean;
```

```
    message: string;

    nextStage?: number |
null;
}

export interface HintResponse {

    hint: string;

    hintsRemaining:
number;
}
```

6. Code Walkthrough

Mystery

```
export interface Mystery {  
  
  id: number;  
  
  slug: string;  
  
  title: string;  
  
  intro: string;  
  
  totalStages: number;  
  
  currentStage:  
number;  
  
  solved: boolean;  
  
  hintsUsed: number;  
  
  hintsRemaining:  
number;  
  
  reveal: string |  
null;  
  
  currentQuestion:  
string;  
  
  currentOptions:  
string[];  
  
}
```

What it does: Describes the public shape of one mystery.

Why it exists: Every page that shows a mystery uses these fields.

What you see: `reveal` is `string | null` because it is only filled when solved.

In Plain English: "A mystery has these properties, with these types."

Note: `Mystery` does **not** include `stages` or `answer`. Those stay on the server.

Clue

```
export interface Clue {  
  
  stage: number;  
  
  text: string;  
  
}
```

What it does: Describes one clue returned by `GET /api/mysteries/:id/clues`.

Why it exists: The `CluePanel` renders a list of these.

AnswerResponse

```
export interface AnswerResponse {  
  
  correct: boolean;  
  
  message: string;  
  
  nextStage?: number |  
  null;  
  
}
```

What it does: Describes what the server returns when an answer is submitted.

Why it exists: The `GamePage` needs to know if the answer was correct.

What you see: `nextStage` is optional; it is `null` on the final correct answer.

HintResponse

```
export interface HintResponse {  
  
  hint: string;  
  
  hintsRemaining:  
  number;  
  
}
```

What it does: Describes what the server returns when a hint is requested.

Why it exists: The GamePage updates its hint budget from this.

7. Common Mistakes

WRONG: Using `any`.

```
const mystery: any = await getMysteryById(id);  
  
mystery.tilte; // typo, no error
```

WRONG: Adding fields the client never uses.

Do not add `stages` to `Mystery` unless the client reads it. The public type should describe only what the code actually uses.

WRONG: Marking `reveal` as `string`.

```
reveal: string; // wrong, it is null before solving
```

CORRECT:

```
reveal: string | null;
```

8. Important Decisions

- **Why keep `Mystery` narrow?** To match exactly what the server sends and what the client reads.
- **Why `string | null` for `reveal` ?** Because the server sends `null` until solved.
- **Why `nextStage?: number | null` ?** Because it is optional and can be `null` .

9. Project Flow

Component needs a `Mystery`

|

▼

Calls `getMysteryById(id): Promise<Mystery>`

|

▼

`apiRequest<Mystery>` returns typed data

|

▼

TypeScript checks every property access

10. Review Questions

1. Why describe API data with interfaces?
2. What does `Mystery` include? What does it exclude?
3. Why is `reveal` typed as `string | null` ?
4. Why is `nextStage` optional?

11. Glossary

| Term | Meaning |

|---|---|

| Interface | A named description of an object's shape. |

| `string \ | null` | A union type: either a string or null. |

| Optional property | A property marked with `?` that may be missing. |

| Public shape | The data the server chooses to expose. |

12. What You Should Be Able to Explain Now

- Why types describe API responses.
- What each type in `types.ts` represents.
- Why `Mystery` excludes accepted answers.

13. Not Yet

- Loading/error/empty states — Session 3.
-

Session 3 — Loading, Error, and Empty States

Learning Objectives

By the end of this session, the team should understand:

- Why async data needs three states.
- How the project renders each.
- Why these are separate components.

1. The Problem

Fetching data takes time. The server might be slow, or it might fail. A page that shows nothing while waiting looks broken. A page that crashes on error is worse.

The player needs to know what is happening.

2. Familiar Idea

In earlier sessions, we showed "Loading tasks..." while the API was fetching. If it failed, we showed a red error message.

The same three states apply here.

3. The New Concept

Every async data render needs three states:

- **Loading:** waiting for the response.
- **Error:** the request failed.
- **Empty:** the request succeeded but returned nothing.

Each state has its own UI. The project uses three small components.

4. Tiny Example

```
if (isLoading) return <LoadingMessage  
label="Loading ... " />;  
  
if (error) return <ErrorMessage message={error}  
onRetry={reload} />;  
  
if (items.length === 0) return <EmptyState  
message="Nothing here." />;  
  
return <List items={items} />;
```

5. Project Application

File: client/src/components/LoadingMessage.tsx

```
interface LoadingMessageProps {  
  
  label: string;  
  
}  
  
export default function LoadingMessage({ label }:  
LoadingMessageProps) {  
  
  return <p  
role="status">{label}</p>;  
  
}
```

File: client/src/components/ErrorMessage.tsx

```
import type { ReactNode } from "react";

interface ErrorMessageProps {

  message: string;

  onRetry: () =>
void;

  title?: string;

  retryLabel?: string;

  children?:
ReactNode;
}

export default function ErrorMessage({

  message,

  onRetry,

  title,

  retryLabel =
"Try again",

  children,

}: ErrorMessageProps) {

  return (

    <div
role="alert">

      {title
    && <h1>{title}</h1>}

    <p>{message}</p>

    {children}
```

```
        <button type="button"
onClick={onRetry}>

        {retryLabel}

    </button>

</div>

);

}
```

File: client/src/components/EmptyState.tsx

```
interface EmptyStateProps {

    message: string;

}

export default function EmptyState({ message }: EmptyStateProps)
{

    return <p
className="empty-state">{message}</p>;

}
```

File: client/src/pages/MysteriesPage.tsx (excerpt)

```
{isLoading && <LoadingMessage label="Loading
mysteries ... " />}

{!isLoading && error && (

  <ErrorMessage

    message={error}

    onRetry={() =>
setReload((value) => value + 1)}

    title=""

  />

)}

{!isLoading && !error && (mysteries.length
=== 0 ? (

  <EmptyState
message="No mysteries are available yet." />

) : (

  <ul
className="case-list">

    {mysteries.map((mystery) => (

      <li
key={mystery.id}>

        <MysteryCard ... />

      </li>

    ) )}

  </ul>
```

```
}}
```

6. Code Walkthrough

LoadingMessage

```
return <p role="status">{label}</p>;
```

What it does: Renders a simple paragraph.

Why it exists: One place to change the loading text style.

What you see: "Loading mysteries..." while fetching.

In Plain English: "Tell screen readers this is a status message."

Read it as: "Paragraph with a status role."

ErrorMessage

```
<div role="alert">

  {title &&
  <h1>{title}</h1>}

  <p>{message}</p>

  {children}

  <button
  type="button" onClick={onRetry}>

    {retryLabel}

  </button>

</div>
```

What it does: Displays an error with an optional title, optional extra content, and a retry button.

Why it exists: The retry button lets the user try again without reloading the browser.

What you see: A red-bordered panel with a message and a button.

In Plain English: "Announce this error and offer a retry."

Read it as: "Alert role, title, message, children, retry button."

EmptyState

```
return <p  
  className="empty-state">{message}</p>;
```

What it does: Explains that the list is empty.

Why it exists: The player should know the request succeeded but there is nothing to show.

What you see: A muted paragraph.

7. Common Mistakes

WRONG: Only rendering the happy path.

```
return <List items={mysteries} />;
```

If `mysteries` is empty or the fetch failed, the page is broken.

WRONG: Reusing the same component for loading and empty.

They mean different things. "Loading..." is not the same as "Nothing here."

WRONG: Putting the fetch inside the loading check.

```
if (isLoading) {  
  fetch(...); //  
  wrong, this causes infinite loops  
}
```

Fetching belongs in `useEffect`, not in render.

8. Important Decisions

- **Why separate components?** Each state has a different meaning and different styling.
- **Why `role="status"` ?** Screen readers announce loading messages.
- **Why `role="alert"` ?** Screen readers announce errors immediately.
- **Why a retry callback?** The user should not need to refresh the page.

9. Project Flow

Page loads

|

▼

isLoading = true → <LoadingMessage />

|

▼

fetch succeeds → isLoading = false, data set

|

▼

data empty → <EmptyState />

|

▼

data present → render list

|

▼

fetch fails → error set

|

▼

<ErrorMessage onRetry={ ... } />

|

▼

User clicks retry → reload counter increments → effect runs

again

10. Review Questions

1. What are the three async states?
2. Why does the project use separate components?
3. What does `role="status"` do?
4. What does `role="alert"` do?
5. Why does `ErrorMessage` take an `onRetry` callback?

11. Glossary

| Term | Meaning |

|---|---|

| Loading state | What the page shows while waiting. |

| Error state | What the page shows when the request fails. |

| Empty state | What the page shows when the result is empty. |

| `role="status"` | An ARIA role for polite announcements. |

| `role="alert"` | An ARIA role for urgent announcements. |

| Retry | Re-running a failed request. |

12. What You Should Be Able to Explain Now

- Why async data needs three states.
- What each component does.
- Why the retry callback exists.
- How the project handles each state.

13. Not Yet

- The full game state on `GamePage` — that is Mulham's session.
 - Component composition for the game — that is Adham's session.
-